

Synchroniser les pratiques Claude Code dans une équipe de 5+ développeurs

Le problème N×M×P

Dans une équipe de 5 développeurs utilisant 3 outils AI sur 2 OS, vous avez potentiellement 30 configurations différentes. Dans la pratique, après 3 mois, aucun développeur n'a le même CLAUDE.md : Alice a ajouté des règles TypeScript que Bob n'a jamais reçues, Carol a configuré des paths macOS qui cassent sur Linux. Ce n'est pas un problème de discipline. C'est un problème d'architecture de configuration.

Solution : Profile-Based Module Assembly

Au lieu d'un fichier monolithique partagé, la structure recommandée décompose les instructions en modules réutilisables assemblés par profil.

profiles/ ← 1 fichier YAML par développeur

alice.yaml bob.yaml

modules/ ← Instructions atomiques par topic

core-standards.md typescript-rules.md

git-workflow.md macos-paths.md

skeleton/ ← Template avec placeholders

claude.md sync-script.ts ← Assembleur automatique

Un module mis à jour se propage automatiquement à tous les développeurs au prochain sync.

Profil développeur (YAML)

```
# profiles/alice.yaml  name : "Alice"  os : "macos"  tools : [ claude-code , cursor ]  modules :  core : [ core-standards , git-workflow , typescript-rules ]  conditional : [ macos-paths , cursor-rules ]
```

Alice reçoit uniquement les modules dont elle a besoin. Bob sur Linux sans Cursor reçoit un CLAUDE.md différent, généré depuis les mêmes sources.

Résultats mesurés

Testé sur une équipe de 5 développeurs, stack TypeScript/Node.js :

Métrique	Monolithique	Profile-Based	Delta
Taille moyenne CLAUDE.md	380 lignes	185 lignes	-51%
Coût tokens estimé	~8 400 tok	~3 450 tok	-59%
Propagation d'une mise à jour	Manuelle	Automatique	

La réduction de 59% vient du fait que chaque développeur ne charge que les modules pertinents à son environnement, au lieu du fichier complet.

settings.json versionné = règles d'équipe

Les règles partagées (hooks, permissions autorisées) vont dans .claude/settings.json versionné. Les overrides personnels dans .claude/settings.local.json gitignore.

```
// .claude/settings.json (équipe, dans le dépôt)  {    "hooks" : {      "PreToolUse"
```

```
: [ { "matcher" : "Bash" , "hooks" : [ ... ] } ] }
```

Cette séparation garantit que les garderails de l'équipe s'appliquent à tous, tout en laissant chaque développeur personnaliser son environnement local sans risque.

Seuil de rentabilité

La mise en place vaut la peine pour des équipes de 5+ développeurs. En dessous, un CLAUDE.md partagé simple suffit. Le coût de maintenance de l'assembleur (quelques dizaines de lignes TypeScript ou Python) est amorti dès le premier mois sur une équipe de 5+. Pour les équipes plus petites, la stratégie simple reste efficace : un CLAUDE.md commun versionné, un settings.local.json par développeur pour les overrides.